TEST

© JMA 2020. All rights reserved

Introducción

- La creación y el mantenimiento de pruebas para aplicaciones Spring presenta a los desarrolladores un conjunto único de desafíos, que incluyen los siguientes:
  - El marco de Spring debe inicializarse y configurarse
  - La aplicación suele tener dependencias de biblioteca de 3ª partes (almacenamiento persistente, servicios externos, etc.)
  - Las aplicaciones a menudo usan funciones integradas de Spring para sesiones, seguridad, mensajería, etc. Estas pueden ser difíciles de configurar para los desarrolladores que son nuevos en las pruebas de Spring.
  - Las dependencias de la aplicación (es decir, beans) deben configurarse adecuadamente
- Spring Framework proporciona un soporte de primera clase para las pruebas de integración en el módulo spring-test.
- El soporte de pruebas de integración de Spring tiene los siguientes objetivos principales:
  - Para administrar el almacenamiento en caché del contenedor Spring IoC entre la ejecución de la prueba.
  - Proporcionar inyección de dependencia personalizada en las prueba.
  - Proporcionar una gestión de transacciones adecuada a las pruebas de integración.
  - Proporcionar clases base específicas de Spring que ayuden a los desarrolladores a escribir pruebas de integración.

© JMA 2020. All rights reserved

Inyección de dependencias

- Cuando el marco TestContext carga el contexto de la aplicación, opcionalmente se puede configurar instancias de sus clases de prueba a través de Dependency Injection.
- Esto proporciona un mecanismo conveniente para configurar casos de prueba utilizando beans preconfigurados del contexto de la aplicación.
- Uno de los grandes beneficios aquí es que se puede reutilizar contextos de aplicación en varios escenarios de prueba (por ejemplo, para configurar gráficos de objetos administrados por Spring, proxies transaccionales, fuentes de datos, etc.), evitando así la necesidad de duplicar la configuración compleja de dispositivos de prueba para casos de prueba individuales.
- Se debe crear un fichero xml o clase de configuración y asociarla a la clase de prueba mediante la anotación @ContextConfiguration o una clase interna anotada con @TestConfiguration.

© JMA 2020. All rights reserved

@ContextConfiguration

```
public static class IoCTestConfig {
	@Bean
	Servicio getServicio() {
		// return new ServicioImpl();
		return new ServicioMockImpl();
	}
}

@Nested
@ContextConfiguration(classes = IoCTestConfig.class)
class IoCTest {
	@Autowired
	Servicio srv;

	@Test
	void contextLoads() {
		assertEquals("Soy el doble de pruebas", srv.getName());
	}
}
```

© JMA 2020. All rights reserved

@TestConfiguration

```
@Nested
class IoCTest {

    @TestConfiguration
    public static class IoCTestConfig {
        @Bean
        Servicio getServicio() {
            return new ServicioMockImpl();
        }
    }

    @Autowired
    Servicio srv;

    @Test
    void contextLoads() {
        assertEquals("Soy el doble de pruebas", srv.getName());
    }
}
```

© JMA 2020. All rights reserved

Sprint Boot

- Spring Boot proporciona una serie de utilidades y anotaciones para ayudarlo a probar su aplicación.
- El "Starter" spring-boot-starter-test (en test scope) contiene las siguientes bibliotecas:
  - JUnit 5 : El estándar de facto para pruebas unitarias de aplicaciones Java.
  - Spring Test y Spring Boot Test: compatibilidad con pruebas de integración y utilidades para aplicaciones Spring Boot.
  - AssertJ: una biblioteca de aserciones fluyentes.
  - Hamcrest: una biblioteca de objetos coincidentes (también conocidos como restricciones o predicados).
  - Mockito: marco de referencia para dobles de prueba simulación en Java.
  - JSONassert: una biblioteca de aserciones para JSON.
  - JsonPath: XPath para JSON.

© JMA 2020. All rights reserved

Sprint Boot

- Una aplicación Spring Boot es un ApplicationContext de Spring, por lo que no se debe hacer nada especial para probarlo más allá de lo que normalmente haría con un contexto Spring vainilla.
- Spring Boot proporciona la anotación @SpringBootTest, que se puede utilizar como alternativa a la anotación estándar @ContextConfiguration cuando se necesiten las funciones de Spring Boot. De forma predeterminada, @SpringBootTest no iniciará un servidor. Puede usar el atributo webEnvironment de @SpringBootTest para refinar aún más cómo se ejecutan sus pruebas:
  - MOCK(Predeterminado): carga una web ApplicationContext y proporciona un entorno web simulado. Los servidores integrados no se inician cuando se usa esta anotación. Si un entorno web no está disponible en el classpath, este modo recurre de forma transparente a la creación de un entorno no web normal ApplicationContext. Para pruebas basadas en simulacros de su aplicación web, se puede utilizar con @AutoConfigureMockMvc o @AutoConfigureWebTestClient.
  - RANDOM_PORT: Carga un WebServerApplicationContext y proporciona un entorno web real. Los servidores integrados se inician y escuchan en un puerto aleatorio.
  - DEFINED_PORT: Carga un WebServerApplicationContext y proporciona un entorno web real. Los servidores integrados se inician y escuchan en un puerto definido (en application.properties) o en el puerto predeterminado de 8080.
  - NONE: Carga un ApplicationContext pero no proporciona ningún entorno web (simulado o de otro tipo).

© JMA 2020. All rights reserved

Autoconfiguraciones

- @AutoConfigureTestDatabase: Por lo general tendremos una Base de Datos embebida o en memoria, esta anotación, sin embargo, nos permitirá realizar nuestros test contra una base de datos real.
- @AutoconfigureMockMvc: Nos va a permitir añadir un MockMvc a nuestro ApplicationContext, de esta manera podremos realizar peticiones HTTP contra nuestro controlador.
- @AutoConfigureWebTestClient: Nos permite verificar los endpoint del servidor, agrega WebTestClient al contexto.
- @DataMongoTest: para probar aplicaciones MongoDB, que configura un MongoDB incrustado en memoria si el controlador está en las dependencias, configura un MongoTemplate, busca @Document y configura los repositorios.
- @DataRedisTest: facilita la prueba de aplicaciones de Redis. Busca clases de @RedisHash y configura repositorios Spring Data Redis de forma predeterminada.

© JMA 2020. All rights reserved

Archivos de propiedades

- Al igual que hacemos en nuestra aplicación Spring Boot haciendo uso de diferentes perfiles a través de nuestras propiedades, en nuestros test integrados podemos también cargar propiedades específicas (como application-test.yml) con la anotación @ActiveProfiles.

```
@SpringBootTest
@ActiveProfiles("test")
class MyApplicationTests {
```

- Otra manera de poder cargar propiedades en nuestro ApplicationContext, es haciendo uso de @TestPropertySource, con lo que podemos cargar un fichero personalizado (como src/test/resources/test.properties) para nuestros test.

```
@TestPropertySource(locations = "/test.properties")
```

- ActiveProfiles cargar un archivo de propiedades entero, pero en muchas ocasiones, solo necesitamos sobre escribir alguna propiedad. La anotación @SpringBootTest permite establecer el valor dichas propiedades:

```
@SpringBootTest(properties = {"perfil.valor=Valor para pruebas"})
class MyApplicationTests {
```

© JMA 2020. All rights reserved

@DataJpaTest

- Podemos usar anotaciones @DataJpaTest para que las pruebas se centren solo en probar los componentes JPA. @DataJpaTest deshabilitará la configuración automática completa del contexto de la aplicación y, en su lugar, aplicará solo la configuración relevante para los componentes y las pruebas de JPA.
- @DataJpaTest proporciona una configuración estándar necesaria para probar la capa de persistencia:
  - configurando una base de datos en memoria si está disponible en el classpath (H2). Se puede anular con @AutoConfigureTestDatabase(replace = Replace.NONE)
  - configurando Hibernate, Spring Data y DataSource
  - realizando un @EntityScan
  - activando el registro de SQL (spring.jpa.show-sql=true)

© JMA 2020. All rights reserved

TestEntityManager

- Para llevar a cabo operaciones de base de datos, necesitamos algunos registros que ya estén en nuestra base de datos. Para configurar estos datos, podemos inyectar un bean TestEntityManager, una alternativa al JPA EntityManager estándar, que proporciona los métodos comúnmente utilizados al escribir pruebas.
- De forma predeterminada, las pruebas JPA de datos son transaccionales que se anulan al final de cada prueba (auto roll back). Para deshabilitar este comportamiento:

```
@DataJpaTest
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public class TestAppData {
```

- La anotación @AutoConfigureTestEntityManager permite utilizar TestEntityManager sin la anotación @DataJpaTest.

© JMA 2020. All rights reserved

TestEntityManager

- Para agregar una base de datos en memoria H2 al classpath:

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>test</scope>
</dependency>
```

- Para utilizar el TestEntityManager:

```
@DataJpaTest
class ActorRepositoryTest {
    @Autowired
    private TestEntityManager em;
    @BeforeEach
    void setUp() throws Exception {
        em.persist(new Actor(0, "Pepito", "Grillo"));
        em.persist(new Actor(0, "Carmelo", "Coton"));
        em.persist(new Actor(0, "Capitan", "Tan"));
```

© JMA 2020. All rights reserved

TestEntityManager

```
@DataJpaTest
class ActorRepositoryTest {
	@Autowired
	private TestEntityManager em;
	@Autowired
	ActorRepository dao;
	@Test
	void testGetAll_isEmpty() {
		var rslt = dao.findAll();
		assertThat(rslt).isEmpty();  }

	@Nested
	class Con_datos {
		@Autowired
		private TestEntityManager em;

		@BeforeEach
		void setUp() throws Exception {
			em.persist(new Actor(0, "Pepito", "Grillo"));
			em.persist(new Actor(0, "Carmelo", "Coton"));
			em.persist(new Actor(0, "Capitan", "Tan"));
		}
		@Test
		void testGetAll_isNotEmpty() {
			var rslt = dao.findAll();
			assertThat(rslt.size()).isEqualTo(3);
			assertThat(dao.findTop10ByFirstNameStartingWithOrderByLastNameDesc("C").size()).isEqualTo(2);
			assertThat(dao.findByActorIdGreaterThan(2).size()).isEqualTo(1);
		}

		@Test
		void testGetOne() {
			var item = dao.findById(dao.findAll().get(0).getActorId());
			assertThat(item.isPresent()).isTrue();
			assertThat(item.get()).asString().isEqualTo("Actor  [actorId=4, firstName=Pepito, lastName=Grillo, lastUpdate=null]");
		}

	}
```

© JMA 2020. All rights reserved

@MockBean

- La anotación @MockBean permite agregar objetos simulados al contexto de la aplicación Spring. El doble de prueba reemplazará cualquier bean existente del mismo tipo en el contexto de la aplicación. Si no se defino uno del mismo tipo, se agrega uno nuevo. Se puede usar como una anotación de nivel de clase o en campos de clases @Configuration. Es útil en pruebas unitarias y de integración en las que es necesario aislarlas de las dependencias.
- La anotación @Mock solo se usa en clases de prueba para crear un doble de prueba de una clase o interfaz que no participa en la inyección de dependencias.
- La infraestructura de dobles de pruebas, también para @MockBean, la suministra Mockito.

© JMA 2020. All rights reserved

@MockBean

```
@ComponentScan(basePackages = {"com.example"})
class ActorServiceTest {
    @MockBean
    ActorRepository dao;

    @Autowired
    ActorServiceImpl srv;

    @Test
    void testGetAll_isNotEmpty() {
        List<Actor> lista = new ArrayList<>(
            Arrays.asList(new Actor(1, "Pepito", "Grillo"),
                new Actor(2, "Carmelo", "Coton"),
                new Actor(3, "Capitan", "Tan")));

        when(dao.findAll()).thenReturn(lista);
        var rslt = srv.getAll();
        assertThat(rslt.size()).isEqualTo(3);
    }
```

© JMA 2020. All rights reserved

MockMvc

- Se pueden escribir pruebas unitarias sencillas para Spring MVC instanciando un controlador, inyectándole dependencias y llamando a sus métodos. Sin embargo, dichas pruebas no verifican las asignaciones de solicitudes, el enlace de datos, la conversión de mensajes, la conversión de tipos, la validación y tampoco involucran ninguno de los métodos de soporte @InitBinder, @ModelAttribute o @ExceptionHandler.
- MockMvc proporciona soporte para probar aplicaciones Spring MVC. Realiza el manejo completo de solicitudes de Spring MVC, pero a través de objetos de solicitud y respuesta simulados en lugar de un servidor en ejecución. MockMvc se puede usar solo para realizar solicitudes y verificar respuestas.
- También se puede usar a través de @WebTestClient donde MockMvc está conectado como el servidor para manejar las solicitudes. La ventaja de @WebTestClient es la opción de trabajar con objetos de nivel superior en lugar de datos sin procesar, así como la capacidad de cambiar a pruebas HTTP completas de extremo a extremo en un servidor en vivo y usar la misma API de prueba.

© JMA 2020. All rights reserved

MockMvc

- Cuando se use MockMvc directamente para realizar solicitudes, son convenientes las siguientes importaciones estáticas:

```
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultHandlers.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
```

- Se puede configurar MockMvc para probar un controlador específico:

```
MockMvc mockMvc;
@BeforeEach
void setup() {
    this.mockMvc = MockMvcBuilders.standaloneSetup(new DemoController()).build();
}
```

- MockMvc se puede autoconfigurar con la anotación @WebMvcTest.

© JMA 2020. All rights reserved

@WebMvcTest

- @WebMvcTest configura automáticamente la infraestructura Spring MVC y limita los beans escaneados a @Controller, @ControllerAdvice, @JsonComponent, Converter, GenericConverter, HandlerInterceptor, HandlerMethodArgumentResolver, Filter, WebMvcConfigurer y WebMvcRegistrations. No se escanean los @Component, @Service, @Repository y @ConfigurationProperties, aunque estos últimos se pueden incluir con @EnableConfigurationProperties.
- A menudo, @WebMvcTest se limita a un solo controlador y se usa en combinación @MockBean para proporcionar implementaciones simuladas para los colaboradores necesarios. @WebMvcTest también configura automáticamente MockMvc.

```
@WebMvcTest(DemoController.class)
class DemoControllerTests {
    @Autowired
    private MockMvc mockMvc;
    @MockBean
    private DemoService srv;
```

© JMA 2020. All rights reserved

Solicitudes MockMvc

- MockMvc dispone de un api fluyente para realizar solicitudes y verificar respuestas.

```
mockMvc.perform(…).andExpect(…)...andDo(print());
```

- Para realizar solicitudes que utilicen cualquier método HTTP:
  - get(), post(), put(), delete(), multipart(), …
  - .queryParam().header().accept().contentType().content()

```
mockMvc.perform(get("/api/v1/actores/{id}", 1)
    .queryParam("mode", "edit"))
mockMvc.perform(put("/api/v1/actores/{id}", 1)
    .contentType(MediaType.APPLICATION_JSON)
    .content(objectMapper.writeValueAsString(ele)))
```

© JMA 2020. All rights reserved

Expectativas MockMvc

- Se puede definir múltiples expectativas agregando .andExpectAll(...) o encadenando .andExpect() después de realizar una solicitud.
- MockMvcResultMatchers.* proporciona una serie de expectativas, algunas de las cuales están anidadas con expectativas más detalladas.
- Las expectativas se dividen en afirmaciones que verifican las propiedades de la respuesta (por ejemplo, el estado de la respuesta, los encabezados y el contenido) y otras que permiten inspeccionar aspectos específicos de Spring MVC, como qué método de controlador procesó la solicitud, si se generó y manejó una excepción, cuál es el contenido del modelo, qué vista se seleccionó, qué atributos flash se agregaron, etc.

© JMA 2020. All rights reserved

Expectativas MockMvc

- Se puede definir múltiples expectativas agregando .andExpectAll(...) o encadenando .andExpect() después de realizar una solicitud.
- MockMvcResultMatchers.* proporciona una serie de expectativas, algunas de las cuales están anidadas con expectativas más detalladas.
- Las expectativas se dividen en afirmaciones que verifican las propiedades de la respuesta (por ejemplo, el estado de la respuesta, los encabezados y el contenido) y otras que permiten inspeccionar aspectos específicos de Spring MVC, como qué método de controlador procesó la solicitud, si se generó y manejó una excepción, cuál es el contenido del modelo, qué vista se seleccionó, qué atributos flash se agregaron, etc.

© JMA 2020. All rights reserved

Expectativas MockMvc

- status(): Acceso a aserciones de estado de respuesta.
- cookie(): Acceso a aserciones de cookies de respuesta.
- header(): Acceso a aserciones de encabezado de respuesta.
- content(): Acceso a aserciones del cuerpo de respuesta.
- jsonPath(): Acceso a las aserciones del cuerpo de la respuesta mediante una expresión JsonPath para inspeccionar un subconjunto específico del cuerpo.
- xpath(): Acceso a las aserciones del cuerpo de la respuesta mediante una expresión XPath para inspeccionar un subconjunto específico del cuerpo.
- redirectedUrl(): Afirma que la solicitud se redirigió a la URL dada.
- forwardedUrl(): Afirma que la solicitud se reenvió a la URL dada.
- view(): Acceso a aserciones en la vista seleccionada.
- model(): Acceso a aserciones relacionadas con el modelo.
- handler(): Acceso a aserciones para el controlador que manejó la solicitud.
- request(): Acceso a aserciones relacionadas con la solicitud.

© JMA 2020. All rights reserved

Expectativas MockMvc

```
mockMvc.perform(…)
        .andExpectAll(
                status().isOk(),
                content().contentType("application/json"),
                jsonPath("$.size()").value(3)
        );
mockMvc.perform(…)
        .andExpect(status().isCreated())
        .andExpect(header().string("Location", url + "/1"));
mockMvc.perform(…)
        .andExpect(status().isNotFound())
        .andExpect(jsonPath("$.error").value("Not found"));
mockMvc.perform(…)
        .andExpect(content().contentType(MediaType.APPLICATION_XML))
        .andExpect(xpath("/person/ns:link[@rel='self']/@href", ns)
                .string("http://localhost:8080/people"));
```

© JMA 2020. All rights reserved

Salidas de MockMvc

- Muchas veces, al escribir pruebas, es útil volcar los resultados de la solicitud realizada.

  mockMvc…andDo(print())
- Siempre que el procesamiento de solicitudes no provoque una excepción no controlada, el método print() imprime todos los datos de resultados disponibles en System.out. Si se desea que se registren los datos de resultados en lugar de imprimirlos, se puede invocar el método log(), que registra los datos de resultados como un solo mensaje DEBUG en la categoría de registro org.springframework.test.web.servlet.result.
- En algunos casos, es posible que se desee obtener acceso directo al resultado y verificar algo que no se puede verificar de otra manera. Esto se puede lograr agregando .andReturn() después de todas las demás expectativas:

  MvcResult mvcResult = mockMvc.perform(…).andExpect(…).andReturn();

© JMA 2020. All rights reserved

MockMvc

```
@WebMvcTest(DemosResource.class)
class DemosResourceTest {
	@MockBean
	ActorRepository dao;

	@Autowired
	private DemosResource srv;

	@Autowired
	private MockMvc mockMvc;

	@Test
	void testGetInicio() throws Exception {
		assertEquals("{saludo=Hola mundo}", srv.getInicio().toString());
		mockMvc.perform(get("/")).andExpect(status().isOk())
			.andExpect(content().contentType(MediaType.APPLICATION_JSON))
			.andExpect(jsonPath("$.saludo").value("Hola mundo"));
	}

	@Test
	void testGetActores() throws Exception {
		List<ActorDTO> lista = new ArrayList<>(
			Arrays.asList(new ActorDTO(1, "Pepito", "Grillo"),
				new ActorDTO(2, "Carmelo", "Coton"),
				new ActorDTO(3, "Capitan", "Tan")));

		when(dao.findByActorIdIsNotNull(ActorDTO.class)).thenReturn(lista);
		mockMvc.perform(get("/actores")).andExpect(status().isOk())
			.andExpect(content().contentType(MediaType.APPLICATION_JSON))
			.andExpect(jsonPath("$.size()").value(3))
			.andExpect(jsonPath("$[0].nombre").value("Pepito"))
			.andExpect(jsonPath("$[2].nombre").value("Capitan"))
			.andDo(print());
	}
```

© JMA 2020. All rights reserved

TestRestTemplate

- TestRestTemplate es una alternativa conveniente a RestTemplate que es útil en las pruebas de integración. Se puede obtener una plantilla estándar o una que envíe autenticación HTTP básica (con un nombre de usuario y contraseña). La plantilla es tolerante a errores, se comporta de una manera amigable con las pruebas al no generar excepciones en los errores 4xx y 5xx, dichos errores se deben detectar a través del código de estado del ResponseEntity devuelto.
- Se puede crear una instancia TestRestTemplate directamente en las pruebas de integración:

```
private final TestRestTemplate template = new TestRestTemplate();
```

- Si se usa la anotación @SpringBootTest con RANDOM_PORT o DEFINED_PORT, se puede inyectar una configuración completa TestRestTemplate y comenzar a usarla.

```
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
class MySpringBootTests {
	@Autowired
	private TestRestTemplate template;
```

© JMA 2020. All rights reserved

MockRestServiceServer

- MockRestServiceServer es el punto de entrada principal para las pruebas REST del lado del cliente. Se utiliza para pruebas que implican el uso directo o indirecto de RestTemplate, código que usa internamente el RestTemplate. Proporciona una forma de configurar las solicitudes esperadas que se realizarán a través de RestTemplate y las respuestas simuladas para enviar de vuelta, eliminando así la necesidad de un servidor real. La idea es declarar las solicitudes esperadas y proporcionar respuestas "stub" para concentrarse en probar el código de forma aislada (es decir, sin ejecutar un servidor).

```
import static org.springframework.test.web.client.match.MockRestRequestMatchers.*;
import static org.springframework.test.web.client.response.MockRestResponseCreators.*;

RestTemplate restTemplate = new RestTemplate();
MockRestServiceServer server = MockRestServiceServer.bindTo(restTemplate).build();
server.expect(ExpectedCount.manyTimes(),
requestTo("/fake/42")).andExpect(method(HttpMethod.GET)).andRespond(
        withSuccess("{ \"id\" : \"42\", \"nombre\" : \"pepito\"}", MediaType.APPLICATION_JSON));

ActorDTO obj = restTemplate.getForObject("/fake/{id}", ActorDTO.class, 42);
```

© JMA 2020. All rights reserved

@RestClientTest

- Se puede usar la anotación @RestClientTest para probar clientes REST, con código que usa internamente el RestTemplate. De forma predeterminada, configura automáticamente la compatibilidad con Jackson, GSON y Jsonb, configura un RestTemplateBuild y añade soporte para MockRestServiceServer.

```
//@AutoConfigureWebClient(registerRestTemplate=true).
@RestClientTest(RemoteService.class)
class MyRestClientTests {
   @Autowired
   private RemoteService service;
   @Autowired
   private MockRestServiceServer server;
   @Test
   void getDetailsWhenResultIsSuccessShouldReturnDetails() {
      this.server.expect(requestTo("/greet/details"))
         .andRespond(withSuccess("hello", MediaType.TEXT_PLAIN));
      String greeting = this.service.callRestService();
      assertThat(greeting).isEqualTo("hello");
   }
}
```

© JMA 2020. All rights reserved

@JsonTest

- Para probar que la serialización y deserialización JSON de objetos funciona como se esperaba, se puede usar la anotación @JsonTest. @JsonTest configura automáticamente el asignador JSON compatible disponible, que puede ser una de las siguientes bibliotecas:
  - Jackson ObjectMapper, cualquier @JsonComponent y cualquier ModuleJackson
  - Gson
  - Jsonb

© JMA 2020. All rights reserved

@JsonTest

```
@JsonTest
public class MyJsonTests {
   @Autowired
   private JacksonTester<ActorDTO> json;

   @Test
   void serialize() throws Exception {
      var details = new ActorDTO(1, "Pepito", "Grillo");
      assertThat(this.json.write(details)).hasJsonPathNumberValue("@.id");
      assertThat(this.json.write(details)).extractingJsonPathStringValue("@.nombre") .isEqualTo(details.getFirstName());
   }

   @Test
   void deserialize() throws Exception {
      String content = "{\"id\":1,\"nombre\":\"Pepito\",\"apellidos\":\"Grillo\"}";
      assertThat(this.json.parse(content)).isEqualTo(new ActorDTO(1, "Pepito", "Grillo"));
      assertThat(this.json.parseObject(content).getFirstName()).isEqualTo("Pepito");
   }
}
```

© JMA 2020. All rights reserved